Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers

Ahmed Gouda Ismail[2*], Tarek Ahmed Mahmoud[1] and Mohamed S. Mohamed[2]

[1]Computer Engineering Department, Egypt University of Informatics, Cairo, Egypt.
[2*]Computer Engineering and Artificial Intelligence Department, Military Technical College, Cairo, Egypt.

*Corresponding author(s). E-mail(s): a7medgouda1@gmail.com
Contributing authors: tarek.mohamed@eui.edu.eg; mohamedms@mtc.edu.eg

Abstract

Multi-object trackers rely on score thresholds tuned for one detector. When the detector is replaced, retrained, or compressed, its scores shift, and the tuned thresholds silently stop fitting. This paper proposes a training-free self-calibrating control layer (SCL) between an unchanged detector and tracker. SCL splits the logits of recent scores twice with Otsu's method and classifies the stream as clean or noisy, without labels or future frames. Its novelty is selective, host-aware adaptation. On clean streams it keeps the tracker's scores and lowers a threshold only if it would reject the confident detections; on noisy streams it selects and rescores the detections and sets the thresholds. It acts through four published tracker settings and passes on weak detections that continue tracks, so one frozen configuration serves single-stage and two-stage trackers. It was evaluated on seven trackers, four detection sources, and three datasets; changes are relative to the tracker alone. When detector scores were halved, two trackers fell to zero higher order tracking accuracy (HOTA); with SCL both recovered at least 97% of it. With a replacement detector, RT-DETR-L, SCL raised the multiple object tracking accuracy (MOTA) of ByteTrack and BoT-SORT by 12.6% and 16.3% and cut their identity switches by 43% and 47%, respectively. It also raised the HOTA of PD-SORT, unseen during design, by 0.9%, and kept four well-matched trackers within 0.1%. SCL adds on average no more than 2.95 ms per frame on an ordinary processor, so a detector can be replaced or retrained without retuning the tracker.

Keywords: multi-object tracking, tracking by detection, detector confidence calibration, online adaptation, Otsu thresholding, real-time image processing

1 Introduction

Multi-object tracking (MOT) finds every object of interest in a video and gives each one an identity number (ID) that stays the same over time. It is used in surveillance, driving, and drone systems [1], where results are needed with little delay. Most MOT systems follow the tracking-by-detection design [2]: in each frame, a detector returns candidate boxes with confidence scores, and a tracker links them into tracks, one per object [3–5]. The tracker uses a box only if its score passes a fixed threshold. A two-stage tracker, such as ByteTrack, also uses lower-score boxes in a second step to continue unmatched tracks [4]; a single-stage tracker, such as OC-SORT, ignores every box below one threshold [5].

In real systems the detector is often replaced, retrained, or compressed, while the tracker keeps the thresholds that their authors set for another detector. Detector scores are not calibrated across models and conditions [6], so a new detector usually gives scores on a different scale, and nothing in the system notices. The effect can be large: halving the scores of a published detector reduces the higher order tracking accuracy (HOTA) [7] of two well-known trackers to 0, because no detection passes their thresholds (Sect. 5.3). For example, ByteTrack in its official setting starts a track only from a box with a score of at least 0.7; a car that

the original detector scored 0.9 now gets 0.45, so ByteTrack never starts a track, and its HOTA falls from 67.70 to 0. SCL finds from the scores themselves that the confident group now begins lower and lets it reach the tracker again, and ByteTrack reaches 66.17 HOTA again, with no labels and no change to its code. Retuning every tracker for every detector needs labels, which a working system usually does not have. Adaptive-threshold methods are each built into one tracker [8–10], score calibration needs labelled data [6], and adaptive inference methods learn their settings for one model [11].

This paper presents a self-calibrating control layer (SCL) that sits between a detector and a tracker that are both kept fixed (frozen): their code, weights, and stored settings are unchanged. We call the tracker that SCL serves the *host*. The novelty of SCL is *selective, host-aware adaptation*. It is selective because it first decides, from the detector scores alone and without labels, whether the scores of the stream fit the host at all: when most higher scores of recent frames form a clearly confident group, the stream is clean, and the host keeps its scores and, unless they would reject the confident group, its thresholds. Only when many scores are uncertain does SCL take control: it chooses which candidates reach the host, gives them new scores, and sets the host's thresholds for the frame. It is host-aware because every action is expressed in the host's own terms: four published settings of the host, the host contract (Sect. 3.7), tell the layer where the host's thresholds lie and whether it has a low-score step, so the same frozen rules serve single-stage and two-stage trackers.

With halved detector scores, SCL restored two trackers from 0 to about 66 HOTA; with other detectors, it raised the multiple object tracking accuracy (MOTA) of two-stage trackers by up to 24.7 points and cut their identity switches by 43% to 56%, while well-matched strong trackers stayed within 0.06 HOTA points of their own accuracy.

The main contributions are:

1. *Selective adaptation.* A label-free, causal test of detector–tracker compatibility: nested Otsu splits of recent logits and a whole-stream majority rule decide whether to act, so a well-matched system is left as it is and a mismatched one is corrected (Sects. 3.2–3.4).
2. *Host-aware adaptation.* A host contract of four published values through which one fixed configuration sets the thresholds, scores, and matching tolerance of unchanged single-stage and two-stage trackers, with a track-guided recovery of weak detections for trackers that lack a low-score step (Sects. 3.5–3.7).
3. *Evidence for compatibility.* An evaluation of one frozen configuration on seven trackers, four detection sources, and three datasets, with paired bootstrap intervals, comparisons with adaptive thresholds and the published ACT method, a component ablation and a parameter sensitivity analysis (Sect. 5), the recorded development path (Sect. 4), and a cost of about 3 ms per frame on an ordinary processor (central processing unit, CPU) (Sect. 6).

Section 2 presents the datasets, evaluation metrics, and related work. Section 3 describes the proposed SCL framework, while Section 4 details its development and refinement. Section 5 presents the experimental results and evaluation. Finally, Sections 6 and 7 discuss the practical implications and summarize the main findings and future research directions, respectively.

2 Background and Related Work

2.1 Datasets

Three public datasets are used. *MOT17* [12] contains 14 sequences of people filmed with static and moving cameras. As is common, the second half of each of the seven training sequences, called val-half (2,652 frames), is used for evaluation [4], with the published YOLOX-X detections [13] at the two published emission floors, the lowest scores emitted, of 0.01 and 0.1. To study score changes directly, these scores were also changed before the tracker by four transforms that keep their order: cubing (s^3), halving ($0.5\,s$), and logit temperatures $T = 2$ and $T = 0.5$, which divide the logit (log-odds, Sect. 3.1) of each score by T. *KITTI tracking* [14] has 21 training sequences (8,008 frames) recorded from a car and is scored on cars and pedestrians; here the trackers run with two other detectors, YOLOv8n [15] and RT-DETR-L [16], and keep thresholds that were set for other detectors. *UAVDT* [17] contains drone videos of vehicles; its 20 test sequences (16,592 frames) are used with ByteTrack, YOLOv8n, and RT-DETR-L.

MOT17 with four trackers, KITTI, and the score transforms were development data, looked at while SCL was designed. Two predeclared external trackers, PD-SORT and Hybrid-SORT, were chosen in advance and run once with the frozen configuration on the detections released by their authors on MOT17 val-half. The held-out UAVDT test split was also run once, after a written plan. Each external host was checked against

its authors' number: EXACT within 0.2 HOTA points, CLOSE within 1.0.

2.2 Evaluation metrics

HOTA [7], the main metric, balances finding the objects and keeping their identities,

$$\mathrm{HOTA} = \frac{1}{19} \sum_{\alpha} \sqrt{\mathrm{DetA}_\alpha \, \mathrm{AssA}_\alpha}, \tag{1}$$

where DetA_α and AssA_α are the detection and association accuracy at the localization threshold $\alpha = 0.05, 0.10, \ldots, 0.95$.

MOTA [18] and IDF1 [19] are

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}, \tag{2}$$

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}, \tag{3}$$

where FN are missed objects (false negatives), FP false positives, IDS identity switches (an object's ID changes), and GT the number of ground-truth boxes; IDTP, IDFP, and IDFN count boxes after each ground-truth track is matched to at most one output track.

All metrics are computed with TrackEval (commit 12c8791) and the official ground truth [20]; KITTI HOTA is averaged over cars and pedestrians, and on UAVDT all vehicles form one class and predictions in ignored regions are removed. HOTA, MOTA, and IDF1 are in percent, and differences in points.

All comparisons are paired: each tracker runs alone and with SCL on the same sequences and detections. Paired bootstrap resampling of whole sequences [21], 10,000 times with seed 42, gives a 95% interval (2.5–97.5 percentiles) for each difference, which is called significant when its interval does not include zero.

2.3 Related work

Tracking by detection. SORT matches Kalman-filter predictions to new detections by their overlap [3]; ByteTrack added a second step that uses low-score detections to continue tracks [4]; OC-SORT uses one score threshold [5]. Their thresholds were tuned for one detector, usually YOLOX [13]. Many recent trackers improve association with trained or domain-specific parts [22–27], which change or retrain the tracker; SCL keeps both the tracker and the detector unchanged. Seven published trackers serve as hosts in this study, each with the authors' code and published configuration: ByteTrack, BoT-SORT [28], OC-SORT, SparseTrack [29], BoostTrack [30], PD-SORT [31], and Hybrid-SORT [32]. ByteTrack is used with the thresholds that its authors used on MOT17 (official setting) and with the defaults of the Ultralytics library, which also provides BoT-SORT (library setting).

Adaptive thresholds. Adaptive ByteTrack sets ByteTrack's threshold from each frame [8], AdapTrack adapts its matching thresholds [9], and ACT sets the high and low thresholds from the first quartile and the number of the current scores [10]. ConfTrack weights the Kalman update and the matching cost by the score [33], and PlugTrack blends motion predictors with a trained network [34].

Comparison. Table 1 compares SCL with the adaptive-thresholding and score-calibration methods. The adaptive-thresholding methods act in every frame, read only the current frame or each box, and are built into one tracker; calibration acts on every score and needs labelled data. SCL differs in three ways. It is selective: it first decides, from the scores of past frames, whether to act at all, and leaves a well-matched tracker as it is. It is host-aware: it reads the tracker's published settings and serves several unchanged single-stage and two-stage trackers through them. And it needs neither labels nor trained parts; its new scores encode decisions for the tracker's thresholds rather than calibrated probabilities (Sect. 3.4). It is also tested as one frozen configuration under score changes and on external systems, and against ACT on the same detections (Sect. 5.4).

Score calibration and adaptive inference. Detector families such as YOLOX, YOLOv8, and RT-DETR give different score distributions, and calibration methods learn a mapping from labelled data [6]. Otsu's method picks, without labels, the threshold that best splits a histogram into two groups [35]; SCL applies it to the logits of recent scores. Dynamic networks learn their settings for each model [11], and our earlier controller, AC-MOT (submitted), sets the detector operating point from image and track cues. SCL instead reads the scores and acts on the tracker's input and thresholds.

3 Proposed SCL Framework

SCL is a lightweight, rule-based layer operating between a frozen detector and tracker (the host) without modifying their underlying algorithms. Fig. 1 illustrates the five processing steps of SCL for frame t. First, Otsu-based logit partitioning (Step 1) divides recent detection scores into background, uncertain, and confident groups. Next,

Table 1 SCL compared with adaptive-thresholding and score-calibration methods. Each row is one method; the columns give what it changes, what information it uses, whether it needs labels or trained parts, when it acts, and where it runs. ReID: re-identification network.

Method	What is adapted	Information used	Labels or trained parts	When it acts	Where it works
Adaptive thresholding					
Adaptive ByteTrack [8]	high/low score split	scores of the current frame	none	every frame	inside ByteTrack
AdapTrack [9]	matching thresholds	current frame	ReID network	every frame	its own tracker
ACT [10]	high and low thresholds, chosen from preset levels	first quartile and number of the current scores	none	every frame	inside the tracker
ConfTrack [33]	Kalman update and matching cost	score of each box	ReID network	every box	its own tracker
Score calibration					
Küppers et al. [6]	detector scores, mapped to probabilities	labelled data	calibration map	every score	detector output
This paper					
SCL	candidates, scores, thresholds, matching tolerance	logits of past frames; host contract	none	only when the scores are noisy (clean: thresholds capped)	between a detector and unchanged trackers

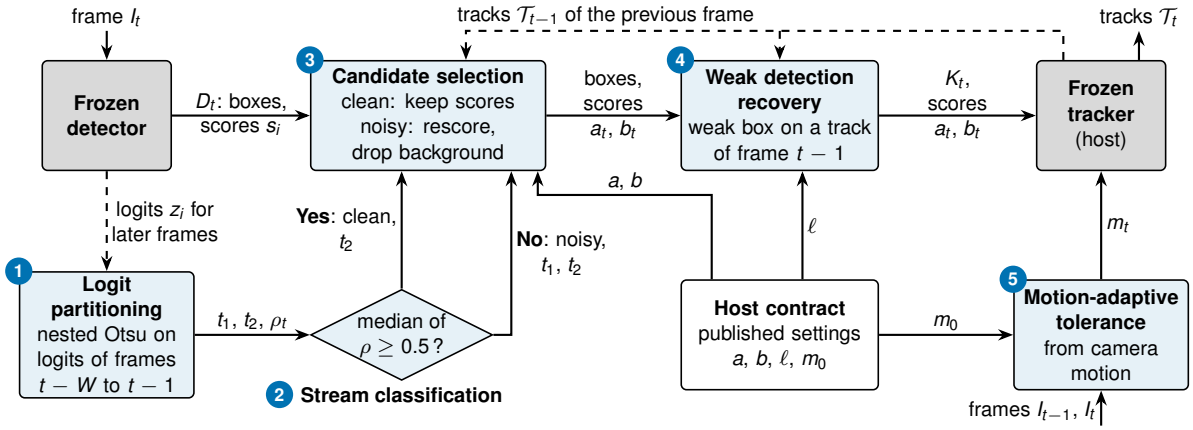


Fig. 1 SCL between a frozen detector and a frozen tracker (the host), for frame t. Each arrow is labelled with the data it carries; dashed arrows carry data kept from earlier frames. The diamond (Step 2) is a decision: the stream is clean (Yes) if the median of ρ is at least 0.5, and noisy (No) otherwise. Steps 1–5 are described in Sects. 3.2–3.6, and the host contract in Sect. 3.7; every symbol is explained in Table 2

detection stream classification (Step 2) determines whether the score distribution is clean or noisy based on historical confidence statistics. Mode-adaptive candidate selection (Step 3) then retains, removes, or remaps detection scores and adjusts the tracker's confidence thresholds according to the operating mode. Track-guided weak detection recovery (Step 4) allows eligible low-confidence detections to support track continuity. Finally, camera motion-adaptive matching tolerance (Step 5) adjusts the association criterion according to estimated camera motion. The host contract provides the tracker's published parameters, while the tracker performs its original association and track-management operations unchanged.

Table 2 lists every symbol of this section in plain words, with its value in a real example, the clean stream of Fig. 2a; the subsections below use the same example, so each number can be followed from the input to the output.

3.1 Inputs and outputs of SCL

In each frame t, the detector gives candidates $D_t = \{(\mathbf{x}_i, s_i, c_i)\}$, each with a box $\mathbf{x}_i$, a score $s_i \in (0, 1)$, and a class c_i. The layer works on the logit

$$z_i = \log \frac{s_i}{1 - s_i}, \tag{4}$$

the log-odds transformation of the confidence score. It maps a score between 0 and 1 to an unbounded scale and keeps the order of the scores: a score of 0.5 gives $z_i = \log 1 = 0$, 0.9 gives $\log(0.9/0.1) = \log 9 = 2.2$, and 0.1 gives $\log(1/9) = -2.2$. The logistic function $\sigma(z) = 1/(1 + e^{-z})$ turns a logit back into a score. The layer uses logits for two reasons. Detector scores crowd near 0 and 1, where small differences in score are large differences in confidence; the logit

Table 2 The symbols of Sect. 3 in plain words. Example: the value in the clean stream of Fig. 2a (MOT17-02, frame 101, ByteTrack in its official setting), as recorded by the frozen SCL; values marked e.g. are worked examples from the text.

Symbol	Meaning	Example
s_i	detector score of candidate i, between 0 and 1	e.g. 0.9
z_i	its logit, $\log(s_i/(1-s_i))$, Eq. (4)	e.g. $\log 9 = 2.2$
W	number of past frames whose logits are pooled	10
Z_t^W	the logits of frames $t-W$ to $t-1$	916 logits
t_1	first split: background lies below it	-0.78 (score 0.32)
t_2	second split: confident lies above it	0.95 (score 0.72)
ρ_t	share of confident candidates in the foreground, Eq. (6)	$175/263 = 0.67$
median ρ	median of ρ over the stream so far; clean if ≥ 0.5	0.63: clean
a, b	the tracker's published scores to match a track and to start one	0.6, 0.7
ℓ	the lowest score the tracker uses	0.1
m_0	the tracker's matching tolerance: 1 minus the smallest IoU that matches	0.8 (IoU ≥ 0.2)
a_t, b_t	the thresholds SCL passes for frame t	0.6, 0.7 (kept)
u_i, $\tilde{s}_i$	rank of s_i among stored scores, and the new score of the noisy mode, Eq. (8)	e.g. $u_i = 0.8 \rightarrow \tilde{s}_i = 0.9$
$\mathcal{T}_{t-1}$	the tracks of the previous frame	–
K_t	the candidates passed to the tracker	–
g_t, r_t	camera shift between two frames; its ratio to the usual shift	e.g. $r_t = 2$
m_t	the tolerance SCL passes for frame t, Eq. (9)	0.8 (clean)

spreads these regions out, so the splits of Step 1 separate the groups more clearly. And a common change of a detector's scores, a change of temperature, divides all logits by the same number; the splits follow this stretch, and each group keeps the same candidates. The host gives its contract $h = (a, b, \ell, m_0)$: the score a needed to match a track, the score b needed to start one, the lowest score ℓ it uses, and its matching tolerance m_0, one minus the smallest intersection over union (IoU) of two boxes that counts as a match. A single-stage tracker has $\ell = \min(a, b)$, a two-stage tracker $\ell < \min(a, b)$. Table 3 (Sect. 3.7) gives these four values for every tracker and the published settings they come from; for ByteTrack in its official setting they are $a = 0.6$, $b = 0.7$, $\ell = 0.1$, and $m_0 = 0.8$. In every frame the layer returns

$$(K_t, \tilde{s}, a_t, b_t, m_t) = \mathcal{L}\left(Z_{<t}, D_t, \mathcal{T}_{t-1}, g_t; h\right): \quad (5)$$

the candidates K_t for the tracker, their scores, and its thresholds and tolerance (the arrows into the tracker in Fig. 1), from the logits $Z_{<t}$ of earlier frames, the tracks $\mathcal{T}_{t-1}$, and the camera motion g_t between the frames I_{t-1} and I_t; no future frame is used.

3.2 Otsu-Based Logit Partitioning and Confidence Estimation

Step 1 in Fig. 1. Input: the logits Z_t^W of the candidates of the last $W = 10$ frames. *Output:* the splits t_1, t_2 and the confident share ρ_t.

The logits are split twice with Otsu's method [35], which picks, from the data alone and without labels or tuning, the threshold that best separates a set of values into two groups; applied twice, it gives the three groups that one threshold cannot. The first split, t_1, separates background (low logits) from foreground; the second, t_2, found on the foreground only, divides it into uncertain and confident candidates. The share of confident candidates in the foreground,

$$\rho_t = \frac{\left|\{z \in Z_t^W : z \geq t_2\}\right|}{\left|\{z \in Z_t^W : z \geq t_1\}\right|}, \quad (6)$$

describes the shape of the score distribution. For example, for the scores 0.02, 0.03, 0.05, 0.4, 0.5, 0.9, and 0.95 (logits -3.9, -3.5, -2.9, -0.4, 0, 2.2, and 2.9), Otsu's method puts the first split at $t_1 = -1.67$ (score 0.16), so the three lowest scores are background, and the second at $t_2 = 1.10$ (score 0.75), so 0.4 and 0.5 are uncertain and 0.9 and 0.95 confident; two of the four foreground scores are confident, so $\rho_t = 2/4 = 0.5$. Fig. 2 shows the same for real streams. In the clean stream of Fig. 2a, the window holds 916 candidates: 653 lie below $t_1 = -0.78$ (score 0.32), 88 between the splits, and 175 above $t_2 = 0.95$ (score 0.72), so $\rho_t = 175/(88 + 175) = 0.67$. In the noisy stream of Fig. 2b, 1,037 are background, 99 uncertain, and 83 confident, so $\rho_t = 83/(99 + 83) = 0.46$. When a detector emits very few low scores, the first split falls inside the objects; since clutter usually gives more candidates than the objects do, a window with fewer candidates below t_1 than above it counts as clean, $\rho_t = 1$ (background check). In both streams of Fig. 2, the background is larger than the foreground (653 against 263, and 1,037 against 182), so the check does not apply.

3.3 Median-Based Detection Stream Classification

Step 2 in Fig. 1. Input: the values ρ of all frames so far. *Output:* the mode of frame t.

The mode describes the distribution of the scores, not whether a single detection is correct. The stream is clean at frame t if the median of all ρ values so far is at least one half, a majority rule,

Table 3 Host contracts: the four published settings of each tracker that SCL reads, and where each value comes from. a: the score a detection needs to be matched to a track; b: the score it needs to start a new track; ℓ: the lowest score the tracker uses at all; m_0: the matching tolerance, one minus the smallest overlap (IoU) that counts as a match. A tracker with a second step for low scores (two-stage) has $\ell < \min(a, b)$; a tracker without it (single-stage) has $\ell = \min(a, b)$. ByteTrack-type trackers publish m_0 directly (match_thresh); SORT-type trackers publish the IoU threshold θ, and $m_0 = 1 - \theta$.

Tracker (setting)	Type	a	b	ℓ	m_0	Published settings behind the values
ByteTrack (official MOT17)	two-stage	0.6	0.7	0.1	0.8	track_thresh 0.6; new tracks need track_thresh + 0.1; second step from 0.1; match_thresh 0.8
ByteTrack (library default)	two-stage	0.25	0.25	0.1	0.8	Ultralytics bytetrack.yaml: track_high_thresh 0.25, new_track_thresh 0.25, track_low_thresh 0.1, match_thresh 0.8
BoT-SORT (library default)	two-stage	0.25	0.25	0.1	0.8	Ultralytics botsort.yaml: the same four values
SparseTrack	two-stage	0.6	0.7	0.1	0.85	track_thresh 0.6; new tracks need track_thresh + 0.1; second step from 0.1; match_thresh 0.85
BoostTrack	two-stage	0.6	0.6	0.1	0.7	det_thresh 0.6; every emitted detection (from 0.1) can be boosted; IoU threshold 0.3
Hybrid-SORT	two-stage	0.6	0.6	0.1	0.75	det_thresh 0.6; low-score step from 0.1; IoU threshold 0.25
OC-SORT	single-stage	0.6	0.6	0.6	0.7	det_thresh 0.6, no low-score step; IoU threshold 0.3
PD-SORT	single-stage	0.6	0.6	0.6	0.7	det_thresh 0.6, no low-score step; IoU threshold 0.3

and noisy otherwise; before the first valid split it is cold. In Fig. 2a, the first frame has no past frames and is cold, and frames 2 to 101 give 100 values of ρ whose median is 0.63, so frame 101 is clean; in Fig. 2b the median of the same 100 frames is 0.42, so it is noisy.

Stable and responsive. The whole-stream median makes the mode stable: a short run of low scores, such as a few frames of blur or occlusion, cannot change it, so the layer does not switch modes in the middle of a stream; during development this rule raised the HOTA of ByteTrack and OC-SORT (Sect. 4.2). The layer still responds quickly to a new score scale: the splits use only the last W frames, so the groups, and in the clean mode the thresholds of Step 3, follow a new scale within W frames, and the recovery of Step 4 works in both modes.

3.4 Mode-Adaptive Detection Candidate Selection and Threshold Adjustment

Step 3 in Fig. 1. Input: D_t, the mode, t_1, t_2, the thresholds a, b, and the tracks $\mathcal{T}_{t-1}$. *Output:* the kept boxes, their scores, and the thresholds a_t, b_t.

Cold and clean modes. The tracker keeps its own thresholds and scores, and only cross-class duplicates are removed. In the clean mode, a threshold so high that it would reject the confident group is lowered to the score $\sigma(t_2)$ at which that group begins:

$$a_t = \min(a, \sigma(t_2)), \qquad b_t = \min(b, \sigma(t_2)). \quad (7)$$

In Fig. 2a, $\sigma(t_2) = 0.72$ is above both $a = 0.6$ and $b = 0.7$, so $a_t = \min(0.6, 0.72) = 0.6$ and $b_t = \min(0.7, 0.72) = 0.7$: the published thresholds are kept. The clean mode is thus a minimal intervention rather than a plain pass-through: the scores are kept, and the output equals that of the host alone whenever the confident group lies above the host's thresholds and no rescue applies, as for BoostTrack, ByteTrack at floor 0.1, and Hybrid-SORT on MOT17 (Sect. 5). When the scores are halved, for example, no score is above 0.5, and ByteTrack in its official setting ($a = 0.6$, $b = 0.7$) never starts a track; the splits move with the halved scores, so $\sigma(t_2)$ falls below 0.5, Eq. (7) lowers both thresholds to it, and the confident candidates reach the tracker again.

Noisy mode. Background candidates ($z < t_1$) are dropped. Each remaining score is replaced by its mid-rank $u_i \in (0, 1)$ among the scores of a remap memory: the share of the stored scores that are lower than s_i, with equal scores counted half. The memory stores the scores kept in the first frame and in every 10th frame, the last 20 of them (before the first sample the current frame is used). The new score is

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\,u_i, & z_i \geq t_2, \\ 0.1 + 0.4\,u_i, & t_1 \leq z_i < t_2, \end{cases} \quad (8)$$

and both thresholds are set to $a_t = b_t = 0.5$. For example, a confident candidate whose score is higher than 80% of the stored scores ($u_i = 0.8$) gets $0.5 + 0.5 \cdot 0.8 = 0.9$, and an uncertain one with $u_i = 0.3$ gets $0.1 + 0.4 \cdot 0.3 = 0.22$. Confident candidates ($z \geq t_2$) then get scores above 0.5 and may start and continue tracks, and uncertain ones ($t_1 \leq z < t_2$) get scores below 0.5 and may only continue tracks through a tracker's low-score step. A weaker candidate that overlaps a stronger one with IoU above one half is removed, unless a different track of the previous frame claims it, which keeps people who stand close together in a crowd.

Score remapping is not probability calibration. Formal calibration learns, from labelled data, a

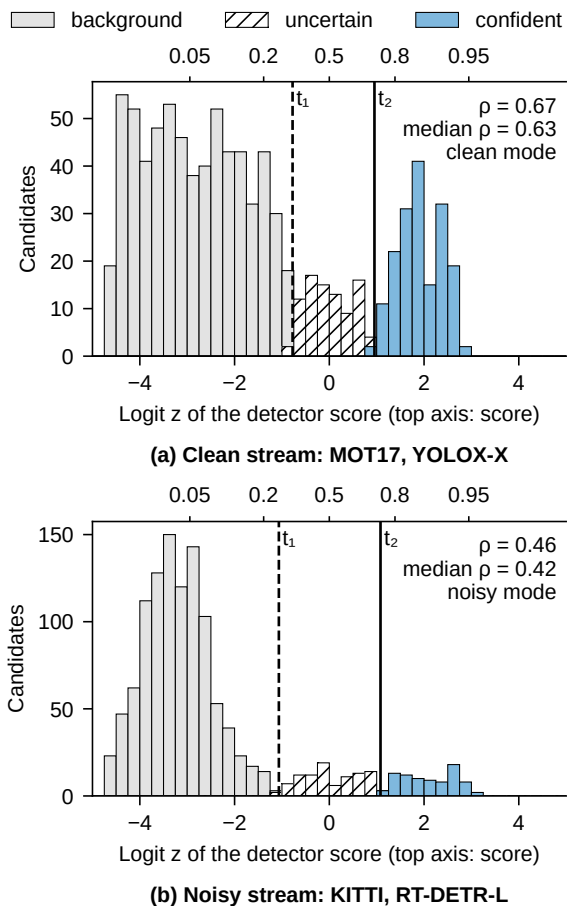


Fig. 2 What SCL sees just before its decision for frame 101: a histogram of the candidates of the last 10 frames. Bottom axis: the logit of the detector score; top axis: the same position as a score; height: the number of candidates. Dashed line: first split t_1; solid line: second split t_2. Light gray bars: background (below t_1); hatched bars: uncertain (between the splits); blue bars: confident (above t_2). ρ: the share of confident candidates in the foreground; the median of ρ decides the mode. (a) MOT17 sequence MOT17-02, published YOLOX-X detections, ByteTrack in its official setting: $\rho = 175/263 = 0.67$ and median $\rho = 0.63 \geq 0.5$, so the stream is clean. (b) KITTI sequence 0001, RT-DETR-L, ByteTrack in its library setting: $\rho = 83/182 = 0.46$ and median $\rho = 0.42 < 0.5$, so the stream is noisy. All values were recorded by the frozen SCL in the runs of Sect. 5

map from a score to the probability that the detection is correct [6]. The remap of Eq. (8) needs no labels and estimates no probability. It keeps the order of the scores within each group and places the groups in fixed ranges, so that the tracker's own thresholds read the decision of the layer: a score above 0.5 means "may start a track", a score between 0.1 and 0.5 "may only continue one". A remapped score of 0.8 therefore says that the candidate ranks high among recent confident candidates, not that it is correct with probability 0.8. Because only ranks are used, the remap itself is unchanged by any order-preserving change of the detector's scores, whereas a learned calibration map would have to be learned again.

3.5 Track-Guided Weak Detection Recovery

Step 4 in Fig. 1. Input: the output of Step 3, the tracks $\mathcal{T}_{t-1}$, and ℓ. *Output:* K_t, their scores, and a_t, b_t for the tracker.

A track can be lost when its object gets low scores for a few frames, above all in single-stage trackers, which have no low-score step. The rescue passes such a candidate, with its score raised just above the lowest threshold of the frame (ℓ for a two-stage tracker, the lower of a_t and b_t for a single-stage one), when it is in the foreground (or, for a single-stage tracker, when there is no background group), overlaps a track of the previous frame with IoU of at least one half, no other usable candidate covers that track, and the tracker could not see it otherwise. For example, for ByteTrack in its official setting (two-stage, $\ell = 0.1$) a rescued candidate gets $0.1 + 0.001 = 0.101$ and enters its low-score step; for OC-SORT in a noisy frame (single-stage, $a_t = b_t = 0.5$) it gets 0.501. It thus gives single-stage trackers the continuation step they lack. The rescue only makes the box visible; the tracker itself still decides, with its own matching, whether and to which track it is assigned.

3.6 Camera Motion-Adaptive Matching Tolerance

Step 5 in Fig. 1. Input: the frames I_{t-1}, I_t and the tolerance m_0. *Output:* the tolerance m_t of frame t.

In noisy frames, the matching tolerance grows with camera motion, because fast motion moves boxes far from their tracks; otherwise $m_t = m_0$. The motion g_t is the shift between consecutive grayscale frames at quarter resolution, found by phase correlation [36], and r_t is its ratio to its median over the last 100 frames ($r_t = 1$ during a warm-up of 5 frames or if this median is zero):

$$m_t = \min\left(0.95,\ 1 - \frac{1 - m_0}{\max(1, r_t)}\right). \qquad (9)$$

For example, with $m_0 = 0.8$ a match needs an IoU of at least $1 - 0.8 = 0.2$. If, in a noisy frame, the camera moves twice as much as usual ($r_t = 2$), then $m_t = \min(0.95, 1 - 0.2/2) = 0.9$, and a match needs an IoU of only 0.1. The cap 0.95 keeps a smallest overlap of 0.05.

Table 4 The fixed values of SCL: each row gives one value, its setting, and where it comes from: a standard value of MOT evaluation, a common threshold, a definition, or a design choice made once at the start of development and never tuned. The same values are used for every tracker, detector, and dataset.

Value	Setting	Source
Window W	10 frames	design choice
Mode rule	clean if median $\rho \geq 0.5$	majority rule
Background check	$\rho = 1$ if fewer below t_1 than above	rule: no background group
Noisy score ranges	$[0.5, 1)$, $[0.1, 0.5)$	common thresholds
Noisy a_t, b_t	0.5	common threshold
Remap memory	20 frames, one every 10	design choice
Duplicate, rescue IoU	0.5	MOT evaluation IoU
Rescued score	lowest threshold $+0.001$	just above the threshold
Motion cue	phase correlation, quarter resolution	standard method [36]
Motion history	100 frames, warm-up 5	design choice
Tolerance cap	0.95	minimum overlap 0.05

3.7 SCL Integration with Existing Trackers

Table 3 lists the contracts; each value is taken from the tracker's published configuration, and none is fitted. For each tracker family, a small adapter links the layer to the tracker block of Fig. 1: it writes the layer's candidates, scores, thresholds, and tolerance into the tracker's existing parameters before each update and then calls the tracker's own update. The matching steps, motion model, track management, and any post-processing of the authors run unchanged, so the host stays frozen: only the values of its thresholds are set for each frame, and the rules are the same for every tracker.

3.8 Fixed SCL Parameters

Table 4 lists all values of SCL. They were fixed before the external evaluation, are the same for every tracker, detector, and dataset, and none was chosen by searching over a range of values. They come from standard values (the IoU of 0.5 of MOT evaluation and the common thresholds 0.5 and 0.1), from definitions, and from design choices kept fixed in all development stages. Algorithm 1 gives Steps 1–5 for one frame.

Algorithm 1 SCL for frame t; the comments name the steps of Fig. 1. The window holds the logits of frames $t-W$ to $t-1$, so the decision for frame t uses no future frame.

Input: candidates D_t, tracks $\mathcal{T}_{t-1}$, motion cue g_t, contract (a, b, ℓ, m_0), constants of Table 4
Output: candidates K_t, scores $\tilde{s}$, thresholds a_t, b_t, tolerance m_t

$z_i \leftarrow$ logit of s_i for all i (Eq. (4))
if no valid split exists **then** (e.g., first frame)
    mode $\leftarrow$ cold; $(a_t, b_t) \leftarrow (a, b)$
else
    $(t_1, t_2) \leftarrow$ nested Otsu on the window (Step 1)
    $\rho_t \leftarrow$ Eq. (6) with the background check
    mode $\leftarrow$ clean if median$(\rho) \geq 0.5$, else noisy (Step 2)
end if
if mode is cold or clean **then** (Step 3)
    remove cross-class duplicates; keep raw scores
    if clean **then** set (a_t, b_t) by Eq. (7)
    end if
else (noisy mode)
    remove duplicates with track context
    drop $z < t_1$; remap scores by Eq. (8)
    $a_t \leftarrow 0.5$; $b_t \leftarrow 0.5$
end if
rescue continuations (not cold) (Step 4)
$m_t \leftarrow$ Eq. (9) if noisy, else m_0 (Step 5)
pass K_t, $\tilde{s}$, a_t, b_t, m_t to the tracker
add the logits left after duplicate removal to the window
add g_t to the motion history
store the tracks of frame t after the tracker update

4 SCL Development and Refinement

This section presents the development and refinement of SCL using the development data described in Sect. 2.1. Section 4.1 introduces the initial always-on banding (AOB) design and discusses its limitations. Section 4.2 describes the incremental improvements that addressed these limitations, supported by the experimental comparisons in Table 5. Finally, Sect. 4.3 explains the configuration freezing and validation procedures performed before external evaluation to ensure reproducibility and prevent changes based on subsequent results.

4.1 Initial Design and Limitations

The earlier design AOB computed the same nested Otsu groups but treated every stream as noisy: in every frame it applied the groups, removed every weaker candidate that overlapped a stronger one with IoU above one half, and rescored all candidates by rank. Table 6 compares AOB and SCL with strong published trackers

Table 5 How SCL was built from AOB on development data. Upper part: each row adds one change to the row above; Effect gives the HOTA (%) before and after that change ("A to B") or compared with the tracker alone (host). The number in parentheses after ByteTrack and OC-SORT is the lowest emitted detector score. Lower part: versions that were tested and not kept, with the reason.

Step	Change	Effect
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ mode gate	mode statistic, host-anchored clean mode, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream mode	mode from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)
Rejected version		*Evidence*
no candidates at frame 1		−0.3 to −0.5 HOTA on every MOT17 host
rescue below a two-stage host's low stage		FP +700 to +1000, −0.4 to −0.5 HOTA
rank-remapped scores in clean frames		ByteTrack −0.2 / −1.6 HOTA
host-clipped noisy primary band		KITTI YOLOv8n ByteTrack +1.4 HOTA but uncontrolled false tracks on a noisy aerial stream (26.9 vs 14.9 boxes/frame)

Table 6 MOT17 val-half with the published YOLOX-X detections: the earlier design AOB and the final SCL, each added to the same strong tracker. Host: the tracker alone; Host+AOB and Host+SCL: the same tracker with each layer, on the same detections. Change: the run with the layer minus Host, with its 95% bootstrap interval in brackets; a negative HOTA, MOTA, or IDF1 change means the layer made the tracker worse; bold: the interval does not include zero, so the change is reliable. ↑: higher is better.

Metric	Host	Host+AOB	AOB change [95% CI]	Host+SCL	SCL change [95% CI]
SparseTrack [29]					
HOTA↑	68.88	64.72	**−4.15 [−5.54, −1.66]**	68.93	+0.06 [−0.01, +0.25]
MOTA↑	77.85	71.71	**−6.14 [−8.10, −2.74]**	77.93	+0.08 [−0.03, +0.30]
IDF1↑	81.97	77.49	**−4.49 [−6.19, −1.96]**	82.13	+0.16 [−0.02, +0.70]
BoostTrack, online [30]					
HOTA↑	68.49	62.61	**−5.89 [−7.76, −2.87]**	68.49	0 (identical)
MOTA↑	75.50	66.64	**−8.87 [−12.37, −7.14]**	75.50	0 (identical)
IDF1↑	81.41	75.19	**−6.22 [−9.33, −2.85]**	81.41	0 (identical)
BoostTrack, with GBI [30]					
HOTA↑	71.73	66.35	**−5.37 [−7.30, −2.75]**	71.73	0 (identical)
MOTA↑	81.03	72.30	**−8.73 [−12.59, −6.50]**	81.03	0 (identical)
IDF1↑	84.16	78.51	**−5.65 [−8.96, −2.77]**	84.16	0 (identical)

on MOT17. AOB lowered the HOTA of SparseTrack by 4.15 points and of BoostTrack by 5.89 points online and 5.37 points with its gap-filling post-processing (GBI), for two reasons: removing overlapping boxes deleted people hidden behind others in crowds, and its thresholds were stricter than the trackers' own. With the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left both BoostTrack versions identical to the host. A layer that acts in every frame therefore cannot be placed safely in front of an unknown tracker, and the next steps let the layer decide from the scores whether to act at all.

4.2 Incremental Design Improvements

Each step was compared with the step before it (Table 5). The *mode gate* added the mode check, the clean mode that keeps the tracker's scores and thresholds, and the crowd-safe duplicate rule; it restored SparseTrack from 64.72 to 68.93 HOTA and BoostTrack from 62.61 to 68.37. The *whole-stream mode* removed mode changes in the middle of streams and raised ByteTrack from 67.561 to 67.702 and OC-SORT from 65.697 to 66.071. The *continuation rescue* added 0.2 to 0.4 points for OC-SORT. The *background check* completed SCL: ByteTrack and BoostTrack returned to output identical to the host, and OC-SORT rose to 67.04 and 66.91 at the two floors. Four other variants were tested and not kept.

4.3 Configuration Freezing and Validation

The final configuration was frozen before any external test as a version-control tag, protected by a lock file with a cryptographic hash of the layer, its configuration, and the evaluation scripts. Sixty-one automated tests check, among others, that the layer uses no future data and passes clean streams unchanged. All results of Sect. 5 come from this frozen configuration; the comparisons, the ablation, and the sensitivity analysis were run after the freeze, through run-time options of the evaluation scripts, and changed none of its values.

5 Experimental Results and Evaluation

This section evaluates the effectiveness and generalization of SCL against the host trackers using identical detections and published tracker configurations. Section 5.1 examines tracking performance across different trackers on MOT17, while Sect. 5.2 evaluates detector–tracker compatibility on KITTI. Section 5.3 investigates robustness to detection confidence-score shifts, and Sect. 5.4 compares SCL with adaptive thresholding methods. Section 5.5 presents component ablation and parameter sensitivity analyses. Finally, Sects. 5.6 and 5.7 assess generalization to previously unseen trackers and the held-out UAVDT dataset, respectively. In all comparisons, the baseline is the host tracker operating without SCL, and Fig. 3 summarizes the HOTA differences across the evaluated configurations.

How to read the result tables. Each result table compares two runs on the same detections: A, the tracker alone (Host), and B, the same tracker with SCL (Host+SCL). HOTA, MOTA, and IDF1 are percentages, and higher is better (↑); IDS counts identity switches, and lower is better (↓). The column Change gives B minus A, so a positive change in HOTA, MOTA, or IDF1 and a negative change in IDS mean that SCL helped. The brackets give the 95% bootstrap interval of the change: when the whole interval lies on one side of zero, the change is reliable and printed in bold; when it includes zero, A and B are not reliably different. "Identical" means that SCL did not change the tracker's output at all. For example, in Table 8 ByteTrack with RT-DETR-L has a MOTA of 47.27 alone and 53.24 with SCL; the change is +5.98 points, and its interval $[+1.16, +9.94]$ lies above zero, so the gain is reliable. Values are rounded separately, so B minus A can differ from the change in the last digit.

5.1 Performance Across Trackers on MOT17

Table 7 shows the development trackers on MOT17 val-half. The two-stage trackers with well-matched detections kept the accuracy of their hosts: BoostTrack, online and with GBI, and ByteTrack at floor 0.1 gave exactly the same output, and SparseTrack rose by 0.06 HOTA points, the self-limiting clean mode. The single-stage OC-SORT improved at both floors (HOTA from 66.43 to 67.04 and from 66.44 to 66.91, MOTA +1.27 and +1.26), with fewer identity switches and higher HOTA on 7 of 7 sequences, through the continuation rescue.

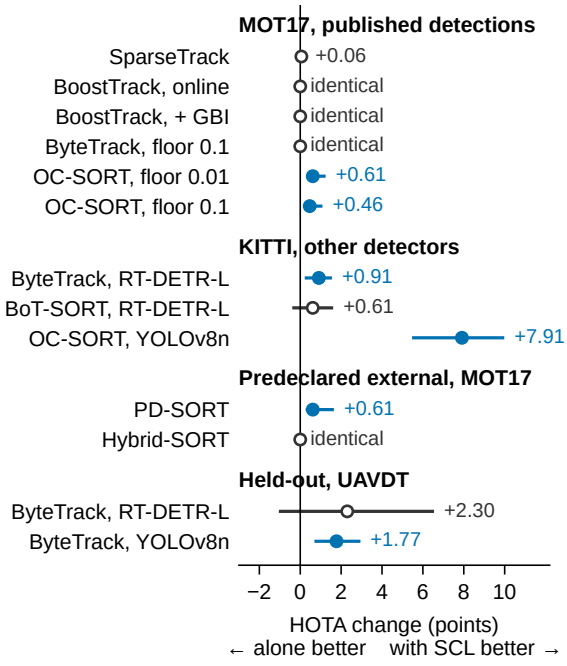


Fig. 3 Summary of the HOTA results: for the tested trackers and detectors, how much SCL changes HOTA (tracker with SCL minus tracker alone, in points). Dot and number: the change; line: its 95% bootstrap interval. Right of the vertical line at zero, SCL is better. Filled dot: a reliable gain (the whole interval is above zero); open dot: no reliable change (the interval includes zero) or identical output

5.2 Cross-Detector Evaluation on KITTI

Table 8 shows what happens when the detector is replaced. With the noisy RT-DETR-L, ByteTrack gained 5.98 MOTA, 3.87 IDF1, and 0.91 HOTA points, and BoT-SORT 7.86 MOTA, 3.10 IDF1, and 0.61 HOTA points, with 297 and 226 fewer identity switches. With YOLOv8n, OC-SORT rose from 36.27 to 44.18 HOTA (+7.91 points, $[+5.47, +9.98]$), 9.40 MOTA points, and 9.60 IDF1 points; it followed many more objects, as its missed objects (FN) fell by 5,310. Fig. 4 follows this cell over three frames: each object keeps its ID, and at frame 102 OC-SORT alone tracked 6 of the 10 labelled objects and with SCL 9.

5.3 Robustness to Detection Score Shifts

Table 9 shows the trackers under the four score transforms. Halving the scores reduced ByteTrack (official setting) and OC-SORT to 0 HOTA; with the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60; BoostTrack rose from 61.33 to 64.84. With $T = 2$, the layer raised ByteTrack and OC-SORT by 1.11 and 1.24 points.

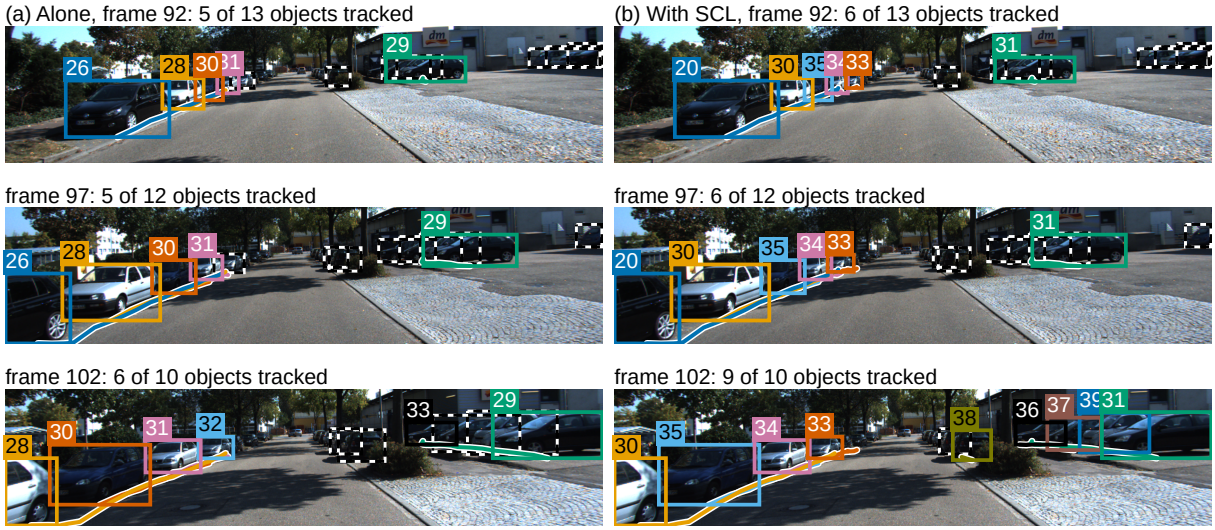


Fig. 4 The same video tracked twice: KITTI sequence 0001 [14] with OC-SORT and YOLOv8n, at frames 92, 97, and 102 (rows). (a) The tracker alone; (b) the same tracker with SCL. Each colour and number is one track (one object identity); its line shows where the object was in the previous 20 frames. Dashed boxes: labelled objects that no track follows (missed). At frame 102, the tracker alone follows 6 of the 10 labelled objects and with SCL 9. IDs are numbered separately in each run. This sequence has the median HOTA gain of the 21 sequences, and frame 102 is its first frame with the largest gain in tracked objects

Table 7 MOT17 val-half with the published YOLOX-X detections: each development tracker alone and with SCL. Host: the tracker alone; Host+SCL: the same tracker with SCL, on the same detections. Change: Host+SCL minus Host, with its 95% bootstrap interval in brackets; bold: the interval does not include zero, so the change is reliable. ↑: higher is better; ↓: lower is better (IDS: identity switches, a count). Paper: the number reported by the tracker's authors (–: not reported). Floor: the lowest detector score emitted.

Metric	Paper	Host	Host +SCL	Change [95% CI]
SparseTrack [29]				
HOTA↑	69.2	68.88	68.93	+0.06 [−0.01, +0.25]
MOTA↑	76.8	77.85	77.93	+0.08 [−0.03, +0.30]
IDF1↑	81.4	81.97	82.13	+0.16 [−0.02, +0.70]
BoostTrack, online [30]				
HOTA↑	68.37	68.49	68.49	0 (identical)
MOTA↑	75.56	75.50	75.50	0 (identical)
IDF1↑	81.35	81.41	81.41	0 (identical)
IDS↓	118	113	113	0 (identical)
BoostTrack, + GBI [30]				
HOTA↑	71.33	71.73	71.73	0 (identical)
MOTA↑	80.55	81.03	81.03	0 (identical)
IDF1↑	83.84	84.16	84.16	0 (identical)
IDS↓	106	97	97	0 (identical)
ByteTrack, floor 0.1 [4]				
HOTA↑	–	67.70	67.70	0 (identical)
MOTA↑	76.6	77.60	77.60	0 (identical)
IDF1↑	79.3	79.47	79.47	0 (identical)
IDS↓	–	214	214	0 (identical)
OC-SORT, floor 0.01 [5]				
HOTA↑	66.5	66.43	67.04	**+0.61 [+0.36, +1.24]**
MOTA↑	74.9	74.67	75.94	**+1.27 [+0.17, +3.01]**
IDF1↑	77.7	78.05	78.71	+0.66 [−0.25, +1.60]
IDS↓	–	211	203	**−8 [−15, −2]**
OC-SORT, floor 0.1 [5]				
HOTA↑	66.5	66.44	66.91	**+0.46 [+0.27, +1.09]**
MOTA↑	74.9	74.67	75.93	**+1.26 [+0.37, +3.34]**
IDF1↑	77.7	78.05	78.33	**+0.287 [+0.003, +0.838]**
IDS↓	–	213	199	−14 [−38, +6]

Table 8 KITTI tracking training with detectors that the trackers were not tuned for (HOTA averaged over cars and pedestrians). Host: the tracker alone; Host+SCL: the same tracker with SCL, on the same detections. Change: Host+SCL minus Host, with its 95% bootstrap interval in brackets; bold: the interval does not include zero, so the change is reliable. ↑: higher is better; ↓: lower is better (IDS: identity switches, a count). BoT-SORT: sequence 0020 is left out of both runs, because the BoT-SORT code stops with a numerical error on it.

Metric	Host	Host +SCL	Change [95% CI]
ByteTrack [4], *RT-DETR-L*			
HOTA↑	50.73	51.64	**+0.91 [+0.22, +1.55]**
MOTA↑	47.27	53.24	**+5.98 [+1.16, +9.94]**
IDF1↑	64.98	68.86	**+3.87 [+1.67, +5.06]**
IDS↓	686	389	**−297 [−530, −122]**
BoT-SORT [28], *RT-DETR-L* (20 sequences)			
HOTA↑	53.91	54.52	+0.61 [−0.39, +1.61]
MOTA↑	48.25	56.11	**+7.86 [+2.21, +13.44]**
IDF1↑	67.74	70.85	**+3.10 [+1.00, +4.57]**
IDS↓	476	250	**−226 [−369, −120]**
OC-SORT [5], *YOLOv8n*			
HOTA↑	36.27	44.18	**+7.91 [+5.47, +9.98]**
MOTA↑	33.99	43.39	**+9.40 [+1.10, +12.30]**
IDF1↑	50.46	60.05	**+9.60 [+5.61, +12.16]**

Across the score changes, the layer raised HOTA significantly in 7 conditions.

5.4 Comparison with Adaptive Thresholding Methods

Two simple baselines set the tracker's matching and birth thresholds in every frame to the first or second Otsu split of the logits of the last 10

Table 9 MOT17 val-half with the detector scores changed before the tracker, while the tracker keeps its published thresholds: s^3 cubes every score, $0.5\,s$ halves it, and $T = 2$ and $T = 0.5$ divide its logit by 2 or by 0.5. HOTA (%, higher is better) of the tracker alone (Host) and with SCL (Host+SCL); 0.00 means that no object was tracked. Change: Host+SCL minus Host, with its 95% bootstrap interval; bold: the interval does not include zero (a reliable change). Lowest emitted score 0.01 (BoostTrack 0.1).

	HOTA (%)		
Score change	Host	Host +SCL	Change [95% CI]
ByteTrack, official setting [4]			
s^3	60.92	65.81	**+4.89 [+2.60, +13.69]**
$0.5\,s$	0.00	66.17	**+66.17 [+49.32, +75.40]**
$T = 2$	65.84	66.95	**+1.11 [+0.35, +3.87]**
OC-SORT [5]			
s^3	58.34	66.60	**+8.26 [+5.86, +16.81]**
$0.5\,s$	0.00	66.72	**+66.72 [+50.32, +75.56]**
$T = 2$	65.99	67.22	**+1.24 [+0.52, +3.42]**
$T = 0.5$	66.61	67.06	+0.45 [−0.01, +0.97]
BoostTrack, online [30]			
s^3	61.33	64.84	**+3.51 [+0.85, +13.28]**
$T = 2$	67.59	67.59	0 (identical)

Table 10 KITTI tracking training: HOTA (%, higher is better) of the tracker alone (Host), of two simple adaptive baselines (t_1, t_2), and of SCL, on the same detections. In the baselines, the tracker's thresholds are set in every frame to the first (t_1) or second (t_2) Otsu split of the logits of the last 10 frames, and the scores are not changed. Halved: halved detector scores. SCL − t_2: the gain of SCL over the stronger baseline, with its 95% bootstrap interval below; bold: the interval does not include zero (a reliable gain).

Cell	Host	t_1	t_2	SCL	SCL − t_2
OC-SORT, YOLOv8n	36.27	39.97	41.09	44.18	**+3.09 [+2.00, +4.03]**
ByteTrack, RT-DETR-L, halved	49.50	49.58	49.50	51.79	**+2.29 [+0.28, +4.96]**
OC-SORT, YOLOv8n, halved	0.00	37.68	43.56	46.30	**+2.74 [+1.32, +3.34]**

frames, t_1 or t_2, and pass the scores unchanged. On KITTI (Table 10), SCL reached the highest HOTA in all three cells, 2.2 to 8.6 points above the baselines, with intervals against the t_2 baseline above zero in every cell, so adaptive thresholds alone do not explain the gains.

Logit-to-score conversion. The baselines and SCL find their splits on logits, while a tracker compares its thresholds with scores, so a split must be converted back with the logistic function: the baselines pass $\sigma(t_1)$ or $\sigma(t_2)$ to the tracker, and SCL passes $\min(a, \sigma(t_2))$ in clean frames (Eq. (7)) and 0.5 with remapped scores in noisy frames. The thresholds recorded in every frame matched these values exactly, with zero difference, in every frame with a split of all eight baseline runs (7,987 frames with RT-DETR-L and 7,940 with YOLOv8n per run) and in all clean and noisy frames of SCL in the four KITTI cells. The baselines therefore received the thresholds they were meant to have.

ACT. ACT [10] is the closest published method: in every frame it sets ByteTrack's high and low thresholds from the first quartile and the number of the current scores, choosing between preset levels. It works on the scores directly, so it needs no conversion, and under halved scores it receives the halved scores, as SCL does. It was run with its published parameters, without retuning, on the same detections and hosts as SCL (Table 11): its high threshold became the host's matching threshold and its low threshold the lower bound of the second step, while the host's birth threshold and matching tolerance stayed unchanged; with OC-SORT, which has one threshold, the high threshold served as that threshold. Before the comparison, the ACT runner was run with the host's own thresholds and reproduced the output of the host alone exactly in all 84 KITTI, 14 MOT17, and 40 UAVDT sequence runs. SCL reached higher HOTA than ACT in every row of Table 11a, significantly under halved scores (+2.01 and +10.06 points) and with YOLOv8n on UAVDT (+1.82), and also in the MOT17 setting for which ACT was designed. On UAVDT (Table 11b), SCL also reached a higher MOTA and had fewer identity switches than ACT with both detectors. With RT-DETR-L its MOTA was 27.99 against 5.34 (+22.65, [+9.03, +47.21]), with 418 fewer identity switches, because the fixed score levels of ACT removed few of the host's false positives.

5.5 Component Ablation and Parameter Sensitivity

Table 12 removes one part of the frozen SCL, or changes one of its values, at a time on the three KITTI cells of Table 10, with the same detections and trackers. Each part matters where its step acts. Without the weak detection recovery, the single-stage OC-SORT lost up to 3.35 HOTA points, as it has no low-score step of its own. Without the noisy mode (always clean), the halved-score cells lost up to 2.44 points, without the remap up to 1.67 points, and without the motion tolerance up to 0.14 points. The clean mode is what protects well-matched trackers: acting in every frame is the earlier design AOB, which lowered the HOTA of strong MOT17 trackers by 4.15 to 5.89 points, while SCL kept them at their own accuracy (Table 6). The mode decision is thus what lets one configuration serve both kinds of stream.

Table 11 ACT [10] and SCL on the same detections and tracker. (a) HOTA in every tested setting. ACT, SCL: HOTA (%, higher is better) of the tracker with each method. SCL − ACT: how much higher SCL is than ACT in HOTA and in IDF1, with its 95% bootstrap interval below. (b) MOTA and identity switches on UAVDT: the value with each method and SCL − ACT with its 95% bootstrap interval in brackets. A positive HOTA, IDF1, or MOTA difference and a negative IDS difference mean that SCL is better; bold: the interval does not include zero (a reliable difference). ↑: higher is better; ↓: lower is better (IDS: identity switches, a count). ACT uses its published parameters, without tuning. KITTI: HOTA averaged over cars and pedestrians; MOT17: val-half, the setting ACT was designed for; UAVDT: test split, one vehicle class, ignored regions removed (Sect. 2.2).

(a) HOTA and IDF1

Data	Detector	Tracker	ACT	SCL	SCL − ACT, HOTA	SCL − ACT, IDF1
KITTI, halved scores	RT-DETR-L	ByteTrack	49.77	51.79	**+2.01** [**+0.01**, **+4.76**]	+1.70 [−1.13, +6.80]
KITTI, halved scores	YOLOv8n	OC-SORT	36.24	46.30	**+10.06** [**+6.88**, **+12.01**]	**+12.03** [**+6.85**, **+14.69**]
MOT17, floor 0.01	YOLOX-X	ByteTrack (official)	67.46	67.68	+0.22 [−0.43, +0.88]	+0.57 [−0.68, +2.12]
MOT17, floor 0.1	YOLOX-X	ByteTrack (official)	67.46	67.70	+0.23 [−0.40, +0.90]	+0.60 [−0.55, +2.10]
UAVDT test	YOLOv8n	ByteTrack	29.01	30.83	**+1.82** [**+0.65**, **+3.00**]	**+4.00** [**+1.85**, **+6.22**]
UAVDT test	RT-DETR-L	ByteTrack	40.14	42.37	+2.23 [−0.97, +6.32]	+5.35 [−0.03, +12.47]

(b) MOTA and identity switches on UAVDT (test split, ByteTrack)

	MOTA↑			IDS↓		
Detector	ACT	SCL	SCL − ACT [95% CI]	ACT	SCL	SCL − ACT [95% CI]
YOLOv8n	16.59	17.29	+0.71 [−1.42, +2.52]	578	393	−185 [−480, +13]
RT-DETR-L	5.34	27.99	**+22.65** [**+9.03**, **+47.21**]	873	455	**−418** [**−718**, **−123**]

Table 12 What each part and each fixed value of SCL contributes, on the three KITTI cells (HOTA, %, higher is better): (a) OC-SORT, YOLOv8n; (b) ByteTrack, RT-DETR-L, halved scores; (c) OC-SORT, YOLOv8n, halved scores. Middle part: one part of SCL removed at a time; the largest HOTA loss it causes, in points, and the cell where it occurs. Lower part: one fixed value changed at a time (frozen value in brackets); the largest HOTA change over the tested values and the three cells, in points, which shows how little the result depends on each value.

Frozen SCL, HOTA: (a) 44.18, (b) 51.79, (c) 46.30		
Part removed	Largest loss	Cell
Weak detection recovery	3.35	(a)
Noisy mode (always clean)	2.44	(c)
Remap (raw scores)	1.67	(b)
Motion tolerance	0.14	(c)
Value changed (frozen)	**Largest change**	**Tested**
Window W (10 frames)	0.56	5, 20
Mode median frames (all)	0.36	last 100, 300
Duplicate/recovery IoU (0.5)	0.62	0.4, 0.6
Motion history (100 frames)	0.09	50, 200

Table 13 The two external trackers, chosen before the run and not used during design (MOT17 val-half, the authors' detections), each run once with the frozen SCL. Host: the tracker alone; Host+SCL: the same tracker with SCL, on the same detections. Change: Host+SCL minus Host, with its 95% bootstrap interval in brackets; bold: the interval does not include zero, so the change is reliable. ↑: higher is better; ↓: lower is better (IDS: identity switches, a count). Paper: the number reported by the authors (for PD-SORT, their released output; –: not reported). EXACT: our Host within 0.2 HOTA of the authors' number; CLOSE: within 1.0.

Metric	Paper	Host	Host +SCL	Change [95% CI]
PD-SORT [31], *EXACT*				
HOTA↑	68.01	68.01	68.62	**+0.61** [**+0.27**, **+1.65**]
MOTA↑	75.19	75.19	76.31	**+1.13** [**+0.27**, **+3.03**]
IDF1↑	81.03	81.03	81.85	**+0.82** [**+0.43**, **+1.94**]
Hybrid-SORT [32], *CLOSE*				
HOTA↑	67.1	66.70	66.70	0 (identical)
MOTA↑	75.8	75.52	75.52	0 (identical)
IDF1↑	78.0	77.56	77.56	0 (identical)
IDS↓	–	259	259	0 (identical)

The sensitivity analysis, planned before the runs, changed the window W, the frames whose median decides the mode, the IoU of the duplicate and recovery rules, and the motion history. For every value and cell, HOTA stayed within 0.62 points of the frozen SCL, and within 0.24 points on the fourth KITTI cell, ByteTrack with the original RT-DETR-L scores. The layer is thus robust to its parameter values: they lie on a flat part of the response, so none of them needs tuning for these detectors and trackers.

5.6 Generalization to Unseen Trackers

Table 13 gives the two predeclared external systems. PD-SORT [31] was reproduced exactly. With SCL, its HOTA rose from 68.01 to 68.62 (+0.61 points, $[+0.27, +1.65]$), its MOTA by 1.13 ($[+0.27, +3.03]$), and its IDF1 by 0.82 ($[+0.43, +1.94]$), with higher HOTA on 7 of 7 sequences; like OC-SORT, it is a single-stage

Table 14 The held-out UAVDT test split (20 drone videos, not used during design), run once with the frozen SCL; ByteTrack with its published setting. Host: the tracker alone; Host+SCL: the same tracker with SCL, on the same detections. Change: Host+SCL minus Host, with its 95% bootstrap interval in brackets; bold: the interval does not include zero, so the change is reliable. ↑: higher is better; ↓: lower is better (IDS: identity switches, a count).

Metric	Host	Host +SCL	Change [95% CI]
ByteTrack, RT-DETR-L [4, 16]			
HOTA↑	40.07	42.37	+2.30 [−1.05, +6.55]
MOTA↑	3.29	27.99	**+24.70 [+10.06, +51.08]**
IDF1↑	48.59	54.33	**+5.74 [+0.13, +13.11]**
IDS↓	1041	455	**−586 [−951, −248]**
ByteTrack, YOLOv8n [4, 15]			
HOTA↑	29.05	30.83	**+1.77 [+0.70, +2.94]**
MOTA↑	16.27	17.29	+1.02 [−1.06, +2.86]
IDF1↑	34.52	38.49	**+3.97 [+1.88, +6.17]**
IDS↓	719	393	**−326 [−691, −73]**

tracker, but it was not seen during design. Hybrid-SORT [32], a two-stage tracker, gave identical output, as predicted before the run.

5.7 Cross-Dataset Generalization on UAVDT

On the held-out UAVDT test split (Table 14), SCL raised the HOTA of the YOLOv8n stream from 29.05 to 30.83 (+1.77 points, [+0.70, +2.94]) and its IDF1 by 3.97 points, and cut identity switches by 45%. With RT-DETR-L, whose host produced many false positives (MOTA 3.29), SCL raised MOTA from 3.29 to 27.99 (+24.70 points, [+10.06, +51.08]) and IDF1 by 5.74 points, and cut false positives by 58% and identity switches by 56%. HOTA rose with both detectors.

6 Discussion and Practical Implications

The score scale is part of the MOT system. Halving the scores reduced ByteTrack and OC-SORT from 67.70 and 66.43 HOTA to 0, and with RT-DETR-L on UAVDT ByteTrack reached a MOTA of only 3.29. The fixed thresholds of a tracker hold only for the detector they were tuned for, so the score scale deserves the same attention as the association method.

Self-limiting by design. In the clean mode the layer keeps the tracker's thresholds and scores, so a well-matched tracker receives its normal input; AOB, which always intervened, lowered the same trackers by 4.15 to 5.89 HOTA points. Single-stage trackers gain through the recovery of weak detections, and two-stage trackers when the detector is noisy.

Deployment. SCL needs no training, no GPU, and no access to the inside of the detector or the tracker. On a CPU it takes 2.95 ms per frame, small next to the 42 ms of the detector, and the tracker itself ran faster with it (1.43 against 2.04 ms per frame); the end-to-end time rose by only 4.5% (4-vCPU Intel Xeon at 2.10 GHz, no GPU, YOLOv8n and ByteTrack on three KITTI sequences, 2,279 timed frames). Because the mode and the confident share ρ need no labels, they can also be logged as a running check of whether a new or retrained detector still fits the tracker it feeds.

Wide applicability. One frozen configuration served seven trackers of both kinds, four detection sources, and three datasets of people and vehicles, filmed from the ground, from a car, and from drones. The splits move exactly with a stretch or shift of the logits, and the layer also restored the trackers under cubed scores (Sect. 5.3), so it follows the common ways in which the score scale of a detector changes.

7 Conclusion

This paper treated the compatibility between a detector's score scale and a tracker's thresholds as a property that can be checked and restored online, without labels and without changing either component. SCL checks it with nested Otsu splits of recent logits and a whole-stream majority rule. It leaves compatible pairs as they are and restores incompatible ones through the tracker's own published settings: it chooses and rescores the candidates, sets the thresholds, and recovers weak detections that continue tracks. The frozen layer was tested on seven trackers and four detection sources. Under halved scores, it restored two trackers from 0 to about 66 HOTA. With a mismatched detector on held-out aerial data, it raised MOTA by up to 24.70 points. Well-matched trackers kept their own accuracy. The layer took 2.95 ms per frame on average on a CPU. Detector–tracker compatibility can therefore be managed as part of the tracking system, so a detector can be replaced or retrained without retuning the tracker. Future work will extend the layer to end-to-end trackers, which learn detection and association jointly. It will also use the label-free mode and confident share as a running check when the detector of a deployed system is updated.

References

[1] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and Tracking Meet Drones Challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7380–7399 (2022) https://doi.org/10.1109/tpami.2021.3119563

[2] Adžemović, M.: Deep Learning-Based Multi-Object Tracking: A Comprehensive Survey from Foundations to State-of-the-Art. The Visual Computer **42**(9), 368 (2026) https://doi.org/10.1007/s00371-026-04607-7

[3] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple Online and Realtime Tracking. In: 2016 IEEE International Conference on Image Processing (ICIP), pp. 3464–3468 (2016). https://doi.org/10.1109/icip.2016.7533003

[4] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In: Computer Vision – ECCV 2022. Lecture Notes in Computer Science, pp. 1–21 (2022). https://doi.org/10.1007/978-3-031-20047-2_1

[5] Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K.: Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9686–9696 (2023). https://doi.org/10.1109/cvpr52729.2023.00934

[6] Küppers, F., Kronenberger, J., Shantia, A., Haselhoff, A.: Multivariate Confidence Calibration for Object Detection. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1322–1330 (2020). https://doi.org/10.1109/cvprw50498.2020.00171

[7] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., Leibe, B.: HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking. International Journal of Computer Vision **129**(2), 548–578 (2021) https://doi.org/10.1007/s11263-020-01375-2

[8] Ma, L.V., Hussain, M.I., Park, J., Kim, J., Jeon, M.: Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. In: 2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS), pp. 370–374 (2023). https://doi.org/10.1109/iccais59597.2023.10382403

[9] Shim, K., Ko, K., Hwang, J., Kim, C.: AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking. In: 2024 IEEE International Conference on Image Processing (ICIP), pp. 2222–2228 (2024). https://doi.org/10.1109/icip51287.2024.10648139

[10] Gwon, D.U., Lee, W.I.: Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking. Electronics **15**(17), 4047 (2026) https://doi.org/10.3390/electronics15174047

[11] Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y.: Dynamic Neural Networks: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7436–7456 (2022) https://doi.org/10.1109/tpami.2021.3117837

[12] Milan, A., Leal-Taixe, L., Reid, I., Roth, S., Schindler, K.: MOT16: A Benchmark for Multi-Object Tracking. arXiv preprint arXiv:1603.00831 (2016). https://doi.org/10.48550/arXiv.1603.00831

[13] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021. arXiv preprint arXiv:2107.08430 (2021). https://doi.org/10.48550/arXiv.2107.08430

[14] Geiger, A., Lenz, P., Urtasun, R.: Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition, pp. 3354–3361 (2012). https://doi.org/10.1109/cvpr.2012.6248074

[15] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0 (2023)

[16] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs Beat YOLOs on Real-Time Object Detection. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/cvpr52733.2024.01605

[17] Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., Tian, Q.: The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking. In: Computer Vision – ECCV 2018. Lecture Notes in Computer Science, pp. 375–391. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-01249-6_23

[18] Bernardin, K., Stiefelhagen, R.: Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics. EURASIP Journal on Image and Video Processing, 1–10 (2008) https://doi.org/10.1155/2008/246309

[19] Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C.: Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking. In: Computer Vision – ECCV 2016 Workshops. Lecture Notes in Computer Science, pp. 17–35 (2016). https://doi.org/10.1007/978-3-319-48881-3_2

[20] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L.: MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. International Journal of Computer Vision **129**(4), 845–881 (2021) https://doi.org/10.1007/s11263-020-01393-0

[21] Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap, (1993). https://doi.org/10.1007/978-1-4899-4541-9

[22] Li, Y., Liu, Y., Zhou, C., Xu, D., Tao, W.: A Lightweight Scheme of Deep Appearance Extraction for Robust Online Multi-Object Tracking. The Visual Computer **40**(3), 2049–2065 (2024) https://doi.org/10.1007/s00371-023-02901-2

[23] Bao, Y., Yu, Y., Qi, Y., Wang, Z.: Multiple Object Tracking with Adaptive Multi-Features Fusion and Improved Learnable Graph Matching. The Visual Computer **40**(4), 2279–2292 (2024) https://doi.org/10.1007/s00371-023-02916-9

[24] Zhang, Y., Liang, Y., Wang, J., Zhu, H., Wang, Z.: Enhanced Multi-Object Tracking via Embedded Graph Matching and Differentiable Sinkhorn Assignment: Addressing Challenges in Occlusion and Varying Object Appearances. The Visual Computer **41**(8), 6067–6085 (2025) https://doi.org/10.1007/s00371-024-03772-x

[25] Ding, Y., Wang, Y., Wang, L., Zhang, B., Li, J., Guo, L., Yang, Z., Liang, W.: Enhancing Multi-Object Tracking Efficiency through Result-Guided Feature Extraction and Query Filtering. The Visual Computer **41**(11), 8153–8167 (2025) https://doi.org/10.1007/s00371-025-03861-5

[26] Liang, Z., Xiao, G., Hu, J., Wang, J., Ding, C.: MotionTrack: Rethinking the Motion Cue for Multiple Object Tracking in USV Videos. The Visual Computer **40**(4), 2761–2773 (2024) https://doi.org/10.1007/s00371-023-02983-y

[27] Sun, Z., Lv, Z., Xiong, M., Hu, W., Wei, G., Zhou, J., He, T., Meng, Q.: Online Visual Multi-Object Tracking via Blind Super-Resolution and Clustering Learning. The Visual Computer **42**(4), 189 (2026) https://doi.org/10.1007/s00371-026-04382-5

[28] Aharon, N., Orfaig, R., Bobrovsky, B.-Z.: BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv preprint arXiv:2206.14651 (2022). https://doi.org/10.48550/arXiv.2206.14651

[29] Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth. IEEE Transactions on Circuits and Systems for Video Technology **35**(5), 4870–4882 (2025) https://doi.org/10.1109/tcsvt.2024.3524670

[30] Stanojevic, V.D., Todorovic, B.T.: BoostTrack: Boosting the Similarity Measure and Detection Confidence for Improved Multiple Object Tracking. Machine Vision and Applications **35**(3) (2024) https://doi.org/10.1007/s00138-024-01531-5

[31] Wang, Y., Zhang, D., Li, R., Zheng, Z., Li, M.: PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues. IEEE Transactions on Consumer Electronics **71**(1), 165–177 (2025) https://doi.org/10.1109/tce.2025.3541839

[32] Yang, M., Han, G., Yan, B., Zhang, W., Qi, J., Lu, H., Wang, D.: Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking. Proceedings of the AAAI Conference on Artificial Intelligence **38**(7), 6504–6512 (2024) https://doi.org/10.1609/aaai.v38i7.28471

[33] Jung, H., Kang, S., Kim, T., Kim, H.: ConfTrack: Kalman Filter-Based Multi-Person Tracking by Utilizing Confidence Score of Detection Box. In: 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 6569–6578 (2024). https://doi.org/10.1109/wacv57701.2024.00645

[34] Kim, S., Lee, S., Cho, M.: PlugTrack: Multi-Perceptive Motion Analysis for Adaptive Fusion in Multi-Object Tracking. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 5709–5716 (2026). https://doi.org/10.1609/aaai.v40i7.37491

[35] Otsu, N.: A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics **9**(1), 62–66 (1979) https://doi.org/10.1109/tsmc.1979.4310076

[36] Reddy, B.S., Chatterji, B.N.: An FFT-Based Technique for Translation, Rotation, and Scale-Invariant Image Registration. IEEE Transactions on Image Processing **5**(8), 1266–1271 (1996) https://doi.org/10.1109/83.506761